

Міністерство освіти і науки України
Національний технічний університет України “КПІ” імені Ігоря
Сікорського
Кафедра обчислювальної техніки ФІОТ

ЗВІТ

з Лабораторної роботи №1
з навчальної дисципліни “Технології Computer Vision
(сертифікаційна програма Data Science із Sigma Software)”

Тема: **ДОСЛІДЖЕННЯ ТЕХНОЛОГІЙ ПОБУДОВИ ТА
ПЕРЕТВОРЕННЯ КООРДИНАТ ПЛОЩИННИХ (2D) ТА
ПРОСТОРОВИХ (3D) ОБ’ЄКТІВ**

Виконав:

Студент 3-го курсу кафедри ОТ ФІОТ,
Навчальної групи ІО-43
Шепітко Владислав
Варіант №17

Перевірів:

Професор кафедри ОТ ФІОТ
Олексій Писарчук

Мета: Виявити дослідити та узагальнити особливості формування та перетворення координат площинних (2d) та просторових (3d) об'єктів.

Завдання: За основу взято III рівень складності - реалізувати розробку програмного скрипта із функціоналом I та II рівнів складності.

- **Вимоги до I рівня складності** - Здійснити синтез математичних моделей та розробити програмний скрипт, що реалізує базові операції 2D перетворень над геометричними примітивами. Для розробки використовувати матричні операції та технології композиційних перетворень. Вхідна матриця координат кутів геометричної фігури має бути розширеною

За варіантом:

17	Реалізувати операції: обертання – (обертання+переміщення). 2. операцію реалізувати циклічно, траєкторію зміни положення цієї операції скрити . Обрати самостійно: бібліотеку, розмір графічного вікна, розмір фігури, параметри реалізації операцій, кольорову гамму усіх графічних об'єктів. Всі операції перетворень мають здійснюватись у межах графічного вікна.	Прямокутник
----	--	-------------

Рис. 1. Завдання для 1-го рівня складності за варіантом

- **Вимоги до II рівня складності** - Здійснити синтез математичних моделей та розробити програмний скрипт, що реалізує базові операції 3D перетворень над геометричними примітивами: аксонометрична проекція будь-якого типу та з циклічне обертання (анімація) 3D графічного об'єкту навколо будь-якої обраної внутрішньої віссю. Траєкторію обертання не відображати. Для розробки використовувати матричні операції. Вхідна матриця координат кутів геометричної фігури має бути розширеною

За варіантом:

17	Динаміка фігури: графічна фігура з'являється та гасне, з'являється в різних частинах графічного вікна змінюючи колір заливки. Обрати самостійно: бібліотеку, розмір графічного вікна, розмір фігури, параметри зміни положення фігури, кольорову гамму усіх графічних об'єктів. Всі операції перетворень мають здійснюватись у межах графічного вікна.	Піраміда з трикутною основою
----	---	------------------------------

Рис. 2. Завдання для 2-го рівня складності

Теоретичні відомості:

Синтезована математична модель для цієї лабораторної роботи складається з наступних базових компонентів:

Матриці базових перетворень. Саме ці матриці описують перетворення точок, в залежності від операції над об'єктом. Загалом ми розрізняємо три типи операцій над об'єктами:

- 1) **Переміщення (нелінійна операція).** Операція переміщення здійснює паралельне перенесення кожної точки об'єкта на задану відстань, що описана вектором $D = [Dx, Dy]$, де Dx - переміщення вздовж осі абсцисс, а Dy - вздовж осі ординат.
- 2) **Обертання (лінійна операція).** Операція обертання виконує поворот точок об'єкту на заданий кут α , навколо початку координат (додатне значення кута α , означає поворот проти годинникової стрілки).

Операцію обертання можна охарактеризувати наступним чином:

$$P' = P \times \begin{bmatrix} \cos x & -\sin x \\ \sin x & \cos x \end{bmatrix}, \text{ де } x - \text{кут повороту.}$$

- 3) **Масштабування (лінійна операція).** Операція масштабування зміщує точки об'єкту на певну відстань від їх початкового розташування. Іншими словами розтягує або стискає об'єкт з певною силою.

Операцію масштабування можна охарактеризувати наступним чином:

$$P' = P \times \begin{bmatrix} Sx & 0 \\ 0 & Sy \end{bmatrix}, \text{ де } Sx \text{ та } Sy - \text{відповідні коефіцієнти масштабування}$$

Якщо Sx або $Sy > 1$, то об'єкт - збільшиться, якщо ж Sx або $Sy < 1$ та > 0 , то об'єкт - зменшиться.

Біля відповідних операцій я зазначив природу їх дій (лінійна чи нелінійна природа операції). Це відіграє важливу роль для подальшого розуміння рішень комп'ютерної реалізації цих операцій над об'єктами.

Суть лінійності полягає у двох факторах:

- **Властивість адаптивності:** $T(u, v) = T(u) + T(v)$
- **Властивість однорідності:** $T(C*u) = C * T(u)$

В нашому випадку це ключові фактори для того, щоб вирішити проблему громіздкості обчислень для базових перетворень об'єктів. Зміщення з самого початку несе нелінійний характер, тому у стандартному вигляді це неефективна операція. Розглянемо випадок, коли наш об'єкт складається з мільйонів вершин. Виходить, що для кожної вершини нам треба окремо враховувати операції масштабування та повороту і додавати зміщення.

Щоб вирішити цю проблему, введемо поняття однорідних координат.

Суть полягає у тому, щоб збільшити розмірність вектора координат на одиницю, додаючи масштабний множник $W \neq 0$. Тобто, якщо раніше ми записували координати, як $[X, Y]$, то тепер ми будемо записувати їх у вигляді $[X, Y, W]$.

Тепер давайте подивимось, як введення однорідних координат вирішило нашу проблему з нелінійною операцією переносу. Операція переносу тепер виглядає наступним чином:

$$P' = P \times \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ Dx & Dy & 1 \end{bmatrix}, \text{ де } P - \text{вектор-рядок нашої координати.}$$

Розглянемо практичний приклад для кращого розуміння. Нехай маємо координати вершини $(3, 5)$. Тепер ми хочемо перенести цю точку на 2 одиниці вздовж осі абсцис та на 3 одиниці вздовж осі ординат. Вийде наступне:

$$\begin{aligned} P' &= [3, 5, 1] \times \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 2 & 3 & 1 \end{bmatrix} \\ &= [3 \times 1 + 5 \times 0 + 1 \times 2 \quad 3 \times 0 + 5 \times 1 + 1 \times 3 \quad 3 \times 0 + 5 \times 0 + 1 \times 1] = [5 \quad 8 \quad 1] \end{aligned}$$

Як бачимо, в результаті отримали вектор-рядок координати з потрібним перенесенням з використанням **ЛІНІЙНОЇ** природи операції (матричне множення).

Для 3D-об'єктів логіка залишається схожою, проте в однорідних координатах, матимемо розмірність вектора координати 4 замість 3 (бо додається вісь аплікату z).

Розібравшись з природою базових операцій над об'єктами на площині та в просторі, залишається тільки зрозуміти як всі ці дії застосувати над точками об'єкту та як спроектувати вершини об'єкту в просторі на площину таким чином, щоб візуально передати об'єм об'єкту на площині проєкції.

Для застосування операцій, ми використовуємо властивість **композиції** матриць. Композиція матриць - це послідовне перемноження матриць трансформацій (матриць обертання, масштабування та переміщення) для отримання однієї результуючої матриці. Рівняння результуючої матриці:

$$M = M_m \times M_o \times M_p, \text{ де}$$

M_m – матриця масштабування, M_o – матриця обертання, M_p – матриця переміщення.

Хочу зазначити, що порядок множення матриць для отримання результуючої матриці перетворень та тип вектору координат **ДУЖЕ ВАЖЛИВІ**. Згадаємо базову властивість матриць, а саме властивість **некомутативності**:

$$A \times B \neq B \times A$$

Наприклад, якщо зробити спочатку зміщення, а потім обертання, то вийде наступний сценарій:

- 1) Зміщуємо об'єкт на певний вектор $[D_x, D_y, 1]$
- 2) Обертаємо об'єкт навколо **ЦЕНТРУ КООРДИНАТ**.

Тобто вийде так, що наш об'єкт буде обертатись по певній дузі навколо початку координат, що не є правильним.

Щодо типу вектору координат. Якщо ми використовуємо вектор-стовпець, то порядок виконання множення матриць буде йти зправа наліво (так як матриця трансформації за замовчуванням має розмірність 4×4 і якщо спробувати перемножити зліва-направо вектор стовпець з розрядністю 4×1 на 4×4 , то таке множення неможливе!), а якщо вектор-рядок, то порядок - зліва-направо.

З приводу проекції координат точок просторового об'єкту на проективну площину. Є два типи проекцій: **ортогональна проекція (паралельна)** та **перспективна проекція**:

- **Ортогональна проекція** - являє собою проекцію тривимірного об'єкта на площину з ортогональними напрямком проекційних променів.
- **Перспективна проекція** - це перетворення одного тривимірного простору в інше, при якому паралельні прямі сходяться, розмір об'єкта зменшується зі збільшенням відстані до центру спостереження, і відбувається неоднорідне спотворення ліній об'єкта, залежне від орієнтації і відстані від об'єкта до центру проекції.

Хочу зазначити, що використання мною перспективної проекції в програмній реалізації було узгоджено з Олексієм Олександровичем на лекційному занятті

Результати виконання лабораторної роботи:

В ході виконання лабораторної роботи, я реалізував інтерактивне середовище для перегляду 3D-об'єктів. Так як перегляд 2D-об'єктів є частковий випадок для 3D ($z = 0$), то фактично дана робота задовольняє вимоги для завдань 1-го та 2-го рівня складностей, а й відповідно 3-го рівня складності також.

Структура проекту:

	.venv	03.09.2026 17:23	Папка с файлами	
	cube.json	04.09.2026 13:49	Исходный файл J...	1 КБ
	engine.py	07.09.2026 15:59	Файл "PY"	12 КБ
	pyramid.json	04.09.2026 14:01	Исходный файл J...	1 КБ
	suzanne.json	07.09.2026 11:19	Исходный файл J...	108 КБ
	Ю-43 Шепітко Влад - Лабораторна ро...	07.09.2026 14:01	Файл "DOCX"	0 КБ

Рис. 3. Структура проекту для виконання Лабораторної роботи №1.

- **.venv** - віртуальне середовище проекту Python.
- **engine.py** - основний файл проекту, де реалізовано усі перетворення над об'єктами та перспективна проекція точок об'єкту на площину проекції (екран)
- **cube.json, pyramid.json, suzanne.json** - файли 3D-об'єктів, що містять матрицю координат вершин об'єкту та ребер.

Результатом виконання програми є інтерактивне середовище перегляду об'єктів з примітивним інтерфейсом, реалізованим за допомогою tkinter та модуля Canvas.

Середовище забезпечує виконання базових операцій над об'єктами за допомогою миші. Управління має наступний вигляд:

- Затискання ЛКМ та перетягування - реалізує обертання об'єкту.
- Затискання колесика миші та перетягування - реалізує переміщення об'єкту.
- Скролл колесика миші - реалізує масштабування об'єкту.

Скріншоти роботи програми:

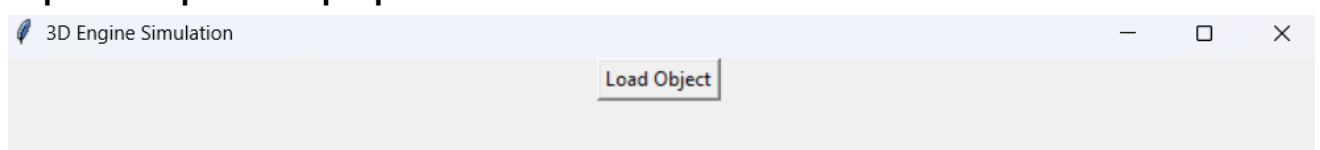


Рис. 4. Вікно завантаження об'єкту для перегляду

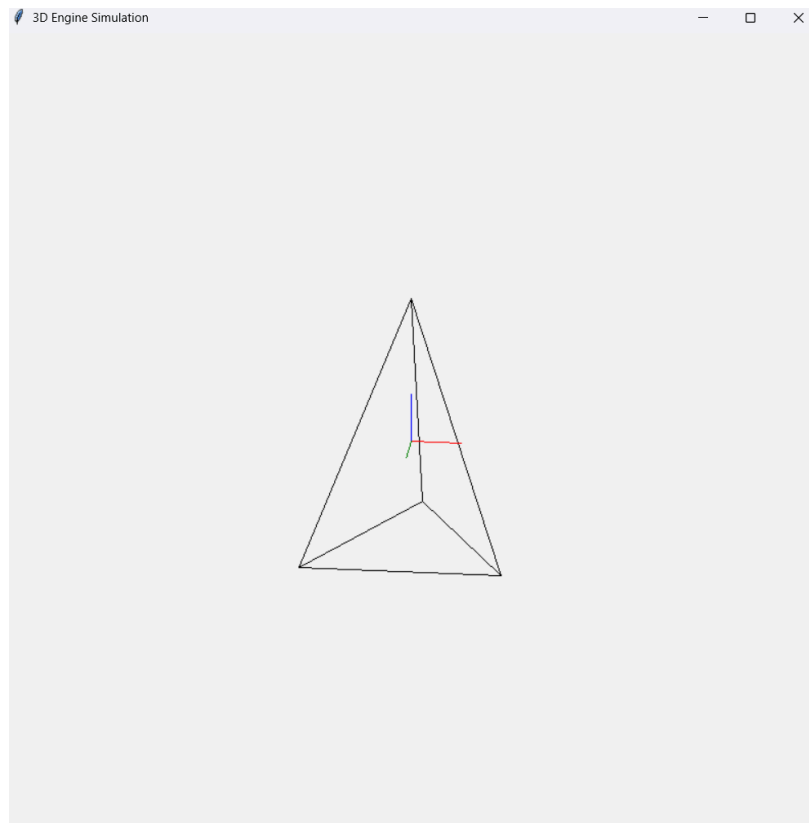


Рис. 5. Вікно інтерактивного середовища перегляду об'єктів.

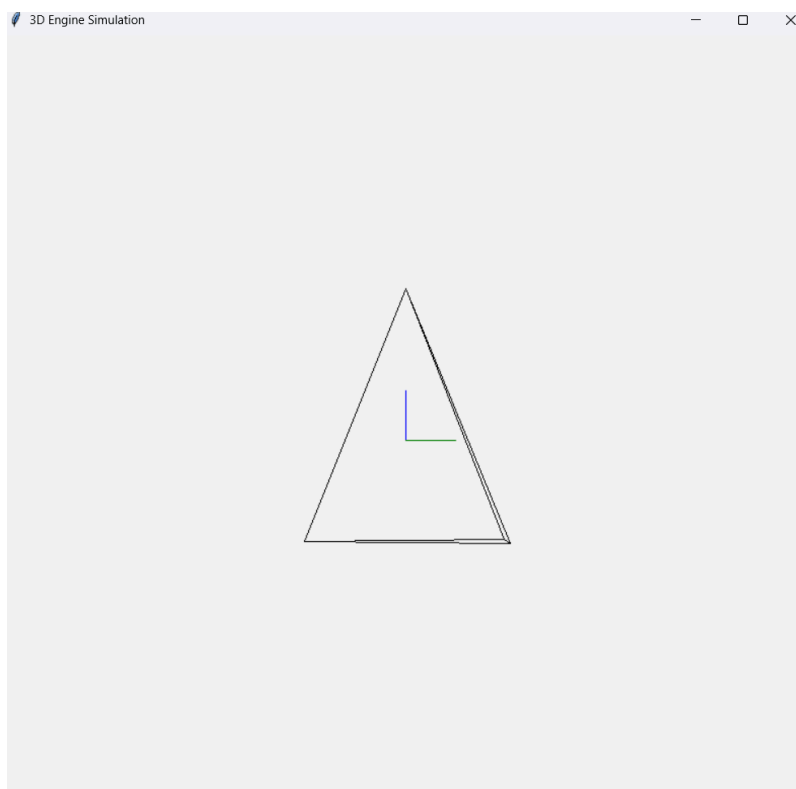


Рис. 6. Виконане обертання трикутної піраміди, навколо осі апікант (бачимо бічну грань піраміди)

Програмний код:

Програма реалізована з використанням наступних бібліотек та модулів:

- Tkinter - для створення простого інтерфейсу інтерактивного середовища
- Tkinter Canvas - вбудований модуль для створення холста для малювання об'єктів
- NumPy - бібліотека для виконання операцій над матрицями та векторами
- json - вбудована Python бібліотека для парсингу JSON файлів та перетворення їх, у об'єкт Python.

Файл *engine.py*

```
import tkinter as tk
from tkinter import filedialog
import numpy as np
import json

class Engine:
    def __init__(self, root):
        '''Ініціалізація головного вікна'''
        self.root = root
        self.root.title("3D Engine Simulation")

        # Налаштування розмірів канвасу
        self.width = 800
        self.height = 800

        # Створення канвасу для відображення графіки
        self.canvas = tk.Canvas(self.root, width=self.width,
height=self.height)
        self.btn = tk.Button(self.root, text="Load Object",
command=self.load_object)
        self.btn.pack()
        self.canvas.pack(fill=tk.BOTH, expand=True)

        # Початкові параметри кутів поворотів об'єкта (ініціалізація
кутів обертання навколо осей X, Y та Z)
        self.angle_x = 0
        self.angle_y = 0
```

```
# Початкові параметри переміщення об'єкта (ініціалізація
координат переміщення об'єкта у 3D просторі)
self.last_x = 0
self.last_y = 0

# Останні координати миші (ініціалізація останніх координат
миші для обробки подій перетягування)
self.last_mouse_x = 0
self.last_mouse_y = 0

# Початкові параметри масштабування об'єкта (ініціалізація
коефіцієнта масштабування об'єкта)
self.scale = 1.0

# Початкові параметри швидкості переміщення
об'єкта (ініціалізація швидкостей переміщення об'єкта у 3D просторі)
self.velocity_x = 0
self.velocity_y = 0

# Початкові параметри швидкості обертання об'єкта (ініціалізація
швидкостей обертання об'єкта у 3D просторі)
self.velocity_rx = 0
self.velocity_ry = 0

self.distance = 500.0 # Відстань від камери до об'єкта для
проекції

self.FOV = 100 # Масштабний коефіцієнт для проекції об'єкта на
2D площину канвасу
self.friction = 0.85 # Коефіцієнт тертя для плавного зменшення
швидкості об'єкта

# Прив'язка обробників подій миші до канвасу
self.canvas.bind("<Button-1>", self.on_mouse_press)
self.canvas.bind("<B1-Motion>", self.on_mouse_drag)
self.canvas.bind("<MouseWheel>", self.on_mouse_scroll)
self.canvas.bind("<Button-2>", self.on_middle_press)
self.canvas.bind("<B2-Motion>", self.on_middle_drag)

def load_object(self):
    '''Завантаження об'єкта з JSON файлу'''
    filepath = filedialog.askopenfilename(filetypes=[("JSON Files",
"*.json")])
    if filepath:
```

```

        with open(filepath, 'r') as file:
            data = json.load(file)
            obj_vertices = data.get("obj_vertices", [])
            obj_edges = data.get("obj_edges", [])
            self.btn.destroy()
            self.scene_init(obj_vertices, obj_edges)
            self.render() # Виклик методу render для відображення
завантаженого об'єкта

    def scene_init(self, obj_vertices=None, obj_edges=None):
        '''Ініціалізація об'єктів сцени'''
        self.obj_vertices = np.array(obj_vertices) # Матриця вершин
об'єкта
        self.obj_edges = np.array(obj_edges) # Матриця ребер об'єкта

        self.axes_vertices = np.array([[0, 0, 0, 1], [0.5, 0, 0, 1],
[0, 0.5, 0, 1], [0, 0, 0.5, 1]]) # Матриця вершин координатних осей
        self.axes_edges = np.array([[0, 1], [0, 2], [0, 3]]) # Матриця
ребер координатних осей

    def update_physics(self):
        '''Оновлення параметрів об'єкта на основі його швидкостей та
тертя (симуляція інерції та плавного зменшення швидкості об'єкта)'''
        # Оновлення координат об'єкта на основі його швидкостей
переміщення
        self.last_x += self.velocity_x
        self.last_y += self.velocity_y

        # Оновлення кутів обертання об'єкта на основі його швидкостей
обертання
        self.angle_x += self.velocity_rx
        self.angle_y += self.velocity_ry

        # Застосування тертя для плавного зменшення швидкостей
переміщення об'єкта
        self.velocity_x *= self.friction
        self.velocity_y *= self.friction

        # Застосування тертя для плавного зменшення швидкостей
обертання об'єкта
        self.velocity_rx *= self.friction
        self.velocity_ry *= self.friction

```

```

def get_transfer_matrix(self, dx, dy, dz):
    '''Обчислення матриці переносу об'єкта у 3D просторі'''
    return np.array([
        [1, 0, 0, 0],
        [0, 1, 0, 0],
        [0, 0, 1, 0],
        [dx, dy, dz, 1]
    ])

def get_rotation_matrix(self, angle_x, angle_y):
    '''Обчислення матриці обертання навколо осей X, Y та Z'''
    rx = np.array([ # Матриця обертання навколо осі X
        [1, 0, 0, 0],
        [0, np.cos(angle_x), np.sin(angle_x), 0],
        [0, -np.sin(angle_x), np.cos(angle_x), 0],
        [0, 0, 0, 1]
    ])

    ry = np.array([ # Матриця обертання навколо осі Y
        [np.cos(angle_y), 0, np.sin(angle_y), 0],
        [0, 1, 0, 0],
        [-np.sin(angle_y), 0, np.cos(angle_y), 0],
        [0, 0, 0, 1]
    ])

    rz = np.eye(4) # Матриця обертання навколо осі Z (ідентична
# матриця, оскільки обертання навколо Z відсутнє на рівні приладу вводу)

    # Повертаємо матрицю, що є результатом композиції матриць
# обертання навколо осей Z, Y та X відповідно
    return rz @ ry @ rx

def get_scale_matrix(self, scale):
    '''Обчислення матриці масштабування об'єкта у 3D просторі'''
    return np.array([
        [scale, 0, 0, 0],
        [0, scale, 0, 0],
        [0, 0, scale, 0],
        [0, 0, 0, 1]
    ])

def project_vertex(self, vertex):
    '''Перетворює 3D координати вершини у 2D координати канвасу'''

```

```

        x_2d = (vertex[0] / (vertex[2] / self.distance + 1)) * self.FOV
+ self.width / 2
        y_2d = (-vertex[1] / (vertex[2] / self.distance + 1)) *
self.FOV + self.height / 2

        return x_2d, y_2d

def render(self):
    '''Відображення об'єкта на канвасі'''
    self.update_physics() # Оновлення параметрів об'єкта перед
відображенням
    self.canvas.delete("all") # Очищення канвасу перед новим
відображенням

    # Отримання матриць трансформацій (масштабування, обертання та
переносу)
    scale_matrix = self.get_scale_matrix(self.scale)
    rotation_matrix = self.get_rotation_matrix(self.angle_x,
self.angle_y)
    transfer_matrix = self.get_transfer_matrix(self.last_x,
self.last_y, 0)

    # Комбінування матриць трансформацій у одну матрицю
    transformation_matrix = scale_matrix @ rotation_matrix @
transfer_matrix

    # Застосування матриці трансформацій до вершин об'єкта
    transformed_vertices = self.obj_vertices @
transformation_matrix

    # Застосування матриці трансформацій до вершин координатних
осей
    transformed_axes_vertices = self.axes_vertices @
transformation_matrix

    # Відображення ребер об'єкта на канвасі
    for edge in self.obj_edges:
        start_vertex = transformed_vertices[edge[0]]
        end_vertex = transformed_vertices[edge[1]]

        # Проекція 3D координат на 2D площину канвасу
        x1, y1 = self.project_vertex(start_vertex)
        x2, y2 = self.project_vertex(end_vertex)

```

```

        # Малювання лінії між двома вершинами ребра
        self.canvas.create_line(x1, y1, x2, y2, fill="black")

axis_colors = ["red", "blue", "green"]

for axe, edge in enumerate(self.axes_edges):
    start_vertex = transformed_axes_vertices[edge[0]]
    end_vertex = transformed_axes_vertices[edge[1]]

    # Проекція 3D координат на 2D площину канвасу
    x1, y1 = self.project_vertex(start_vertex)
    x2, y2 = self.project_vertex(end_vertex)

    # Малювання лінії між двома вершинами ребра координатних
    осей
    self.canvas.create_line(x1, y1, x2, y2,
fill=axis_colors[axe])

    self.root.after(16, self.render) # Виклик методу render через
16 мс для створення ефекту анімації

def on_mouse_press(self, event):
    '''Обробка натискання клавіші миші'''
    self.last_mouse_x = event.x
    self.last_mouse_y = event.y

    self.velocity_rx = 0
    self.velocity_ry = 0

def on_mouse_drag(self, event):
    '''Обробка перетягування клавіші миші'''
    dx = (event.x - self.last_mouse_x) * 0.01
    dy = (event.y - self.last_mouse_y) * 0.01

    self.velocity_rx = dy
    self.velocity_ry = -dx

    self.last_mouse_x = event.x
    self.last_mouse_y = event.y

def on_mouse_scroll(self, event):
    '''Обробка прокручування колеса миші'''

```

```

        if event.delta > 0:
            self.scale *= 1.1
        else:
            self.scale /= 1.1

    def on_middle_press(self, event):
        '''Обробка натискання середньої клавіші миші'''
        self.last_mouse_x = event.x
        self.last_mouse_y = event.y

        self.velocity_x = 0
        self.velocity_y = 0

    def on_middle_drag(self, event):
        '''Обробка перетягування середньої клавіші миші'''
        dx = (event.x - self.last_mouse_x) * 0.01
        dy = (event.y - self.last_mouse_y) * 0.01

        self.velocity_x = dx
        self.velocity_y = -dy

        self.last_mouse_x = event.x
        self.last_mouse_y = event.y

root = tk.Tk()
engine = Engine(root)
root.mainloop() # Запуск головного циклу обробки подій

```

Файл pyramid.json (приклад як саме зберігаються файли об'єктів)

```

{
  "obj_vertices": [
    [0, 1.5, 0, 1],
    [-1, -1, 1, 1],
    [1, -1, 1, 1],
    [0, -1, -1, 1]
  ],
  "obj_edges": [
    [0, 1], [0, 2], [0, 3],
    [1, 2], [2, 3], [3, 1]
  ]
}

```

```
}
```

Висновок: В ході лабораторної роботи ми навчилися синтезувати математичну модель для виконання базових операцій над об'єктами на площині (2D-об'єкти) та над об'єктами у просторі. Також розібрали важливі нюанси у синтезі даної математичної моделі, розглянули типи проекцій просторових об'єктів на проективну площину і навіть реалізували перспективну проекцію об'єктів, що є найбільш наближеним типом проекції до реального світу.

Практична цінність даної роботи полягає у знятті рівня абстракції з сучасних ігрових рушіїв (таких як Unity чи Unreal Engine). Вона наочно демонструє, що 3D-графіка - це структурований потік матричних перемножень. Набутий досвід є фундаментальною базою для подальшої роботи з комп'ютерною графікою на низькому рівні, написання шейдерів та розробки інтерактивних візуальних систем.